

LFQ: Logit-aware Final-block Quantization for Boosting the Generation Quality of Low-Bit Quantized LLMs

Jung Hyun Lee^{*1} June Yong Yang^{*2} Jungwook Choi³ Eunho Yang¹⁴

Abstract

As large language models continue to scale, low-bit weight-only post-training quantization (PTQ) offers a practical solution to their memory-efficient deployment. Although block-wise PTQ is capable of matching the full-precision (FP) baseline on basic language modeling and understanding, its quality is degraded for *generative* tasks—especially at longer responses and extended chains of thought, which is critical in boosting task accuracy. We attribute this shortfall to two factors: (i) the omission of the unembedding layer (the LM head) in block-wise optimization and (ii) the reliance on the mean squared error (MSE) objective. Both factors cause the token probability distribution of the quantized model to misalign with that of the FP model, yielding notable accuracy drops on text generation benchmarks. To rectify the discrepancy, we introduce *Logit-aware Final-block Quantization (LFQ)*, a simple yet effective enhancement to block-wise PTQ that quantizes the final Transformer block by minimizing the cross-entropy between the logits of the FP model and those of its quantized counterpart. By aligning token probabilities at the logit level in the final block, LFQ consistently improves the accuracy of complex generation tasks over state-of-the-art block-wise PTQ across diverse model families, while maintaining parity with FP baselines on language modeling and understanding.

¹Kim Jaechul Graduate School of AI, KAIST, Daejeon, South Korea ²LG AI Research, Seoul, South Korea ³Hanyang University, Seoul, South Korea ⁴AITRICS, Seoul, South Korea. Correspondence to: Jung Hyun Lee <onliwad101@kaist.ac.kr>, June Yong Yang <laoconeth@gmail.com>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

1. Introduction

The evident success of large language models (LLMs) (Grattafiori et al., 2024; Qwen et al., 2025; Team et al., 2025) is largely attributed to their ever-increasing number of parameters (Kaplan et al., 2020). However, the proportionally increasing memory footprint of the model significantly impedes the cost-effective deployment of LLMs. Not only is a large model difficult to fit in commercial devices, but the serving cost of the model also increases sharply with the model size. To this end, quantization has been widely adopted to increase the inference efficiency of LLMs by employing lower precision data types.

Recently, weight-only quantization (Frantar et al., 2023; Lin et al., 2024) has emerged as a particularly attractive methodology due to its high compression ratio and effective preservation of model quality. By quantizing the LLM weights into low-precision but retaining difficult-to-quantize activations in full precision (FP), memory pressure is effectively relieved while reducing the accuracy degradation. Low-bit weight-only quantization is obtained either via quantization-aware training (QAT) or post-training quantization (PTQ). Although Liu et al. (2025) shows that QAT is capable of restoring the degraded accuracy even under sub-4-bit settings, the computational resource required to conduct QAT makes it prohibitively memory-intensive and time-consuming. On the other hand, layer-wise PTQ can be conducted with a relatively small amount of resources, but suffers from model quality degradation.

Block-wise PTQ (Lee et al., 2023; Shao et al., 2024; Cheng et al., 2024; Lee et al., 2025b; Chen et al., 2025) strikes an effective balance between the two ends, achieving effective and efficient degradation recovery. By minimizing the mean squared error (MSE) between the outputs of an FP Transformer block and those of its quantized counterpart, cross-layer dependencies within each transformer block are accounted for, recovering the performance comparable to FP baselines on tasks such as language modeling (e.g. WikiText2 (Merity et al., 2016)) and general natural language understanding (e.g. MMLU (Hendrycks et al., 2021)).

However, relatively little attention has been shed on the

degradation of *generation* quality caused by the low-bit quantization of LLMs. It is particularly alarming considering the increasing trend towards generating longer responses for increased task performance. Emerging large reasoning models (DeepSeek-AI et al., 2025; Aggarwal & Welleck, 2025) scale inference-time compute to produce extended chains of thought, thereby achieving higher accuracy on complex multi-step reasoning tasks. As this trend toward generating more tokens continues in pursuit of increased accuracy, serving costs rise sharply, necessitating a strong demand for an efficient quantization method that can maintain the generation quality of FP baselines.

In this work, we uncover that the standard block-wise PTQ approach—while effective at language modeling and understanding—suffers from the degradation of generation quality. The limitation is attributed to the fact that block-wise PTQ only preserves the quality of output activations of a transformer block, rather than preserving the fidelity of the next-token sampling *distribution*. Specifically, (i) existing block-wise PTQ methods completely ignore the unembedding layer (also known as the LM head), and (ii) rely on the MSE as optimization objective. Even when the MSE between the outputs of a quantized block and its FP counterpart is minimized, the actual probabilities assigned to plausible tokens can be perturbed, producing substantial shifts in distribution. Such misalignment is less observable on language understanding tasks, which does not involve autoregressive generation, but becomes pronounced in long-form generation as compounding probability distortions steer the generation trajectory away from the FP baseline.

Motivated by the observations, we propose *Logit-aware Final-block Quantization (LFQ)*, which enables low-bit quantized LLMs to achieve performance close to FP baselines on text generation tasks such as IFEval (Zhou et al., 2023), GSM8K (Cobbe et al., 2021), MATH500 (Lightman et al., 2023), and AIME (AIME, 2025). Unlike standard block-wise PTQ, LFQ quantizes the final Transformer block by minimizing the cross-entropy loss between the logits of the FP model and those of its quantized counterpart. Specifically, all Transformer blocks from the first to the penultimate are quantized by minimizing the MSE between the outputs of the FP and quantized blocks, while the final block is optimized using cross-entropy at the logit level, aligning token probabilities with the FP model and thereby reproducing the token prediction probabilities of the FP model. Thanks to its simple design, LFQ can be seamlessly applied to existing block-wise approaches. Moreover, LFQ consistently improves the generation quality of block-wise PTQ methods, while maintaining performance comparable to FP baselines on language modeling and understanding tasks.

Our contribution is threefold:

- To the best of our knowledge, we are the first to

show that the conventional block-wise PTQ objective—minimizing MSE at intermediate outputs—does not align with reproducing the token predictions of FP models, thereby inducing non-negligible accuracy gaps between FP baselines and their low-bit quantized counterparts on text generation tasks.

- We propose Logit-aware Final-block Quantization (LFQ), which quantizes the final Transformer block by minimizing the cross-entropy between the logits of the FP model and its quantized counterpart, consistently improving the generation quality of low-bit quantized LLMs across existing block-wise PTQ methods.
- We validate LFQ across diverse models—including instruction-tuned, reasoning, and Mixture of Experts (MoE) models—on text generation benchmarks such as IFEval, GSM8K, MATH500, and AIME. We further evaluate LFQ on WikiText2 and MMLU to ensure that it performs comparably to, and in some cases better than, existing block-wise PTQ techniques on language modeling and understanding tasks as well.

2. Problem Statement

Block-wise PTQ progressively quantizes each Transformer block by minimizing the mean squared error (MSE), from the first to the final block. In this section, we focus our attention on the final block, which is distinctive from the other blocks as it is directly attached to the LM head that produces the token sampling distribution. Below, we provide a brief overview of notations and assumptions used for illustrative purposes throughout this paper.

Let $\mathbf{W}_{\text{FP}}, \mathbf{W}_q \in \mathbb{R}^{c_{\text{in}} \times c_{\text{out}}}$ denote the full-precision (FP) final Transformer block and its quantized counterpart, and let $\mathbf{X} \in \mathbb{R}^{L \times c_{\text{in}}}$ represent the input to the final block, where L is the sequence length. Let $\mathcal{V}$ denote the vocabulary, with size $V = |\mathcal{V}|$. The LM head is then defined as $\mathbf{W}_{\text{Head}} \in \mathbb{R}^{c_{\text{out}} \times V}$. For illustrative purposes only, however, we restrict the vocabulary to $\mathcal{V} = \{t_1, t_2\}$, so that $\mathbf{W}_{\text{Head}} \in \mathbb{R}^{c_{\text{out}} \times 2}$. Unless otherwise specified, we omit the normalization layer between the final block and the LM head for simplicity.

First, to illustrate that minimizing the MSE between the outputs of a FP final Transformer block and its quantized counterpart can adversely affect the generation quality of low-bit quantized LLMs, we consider the case where $c_{\text{out}} = 2$. The final block is quantized by minimizing $\|\mathbf{X}\mathbf{W}_{\text{FP}} - \mathbf{X}\mathbf{W}_q\|_F^2$, yielding $\mathbf{X}\mathbf{W}_q = [0.7, 0.3]$ for $\mathbf{X}\mathbf{W}_{\text{FP}} = [0.8, 0.2]$ as an example. However, it is worth noting the following example:

$$\text{When } \mathbf{W}_{\text{Head}} = \begin{bmatrix} 0.5 & 0.3 \\ 0.5 & 1.0 \end{bmatrix},$$

$$\mathbf{X}\mathbf{W}_{\text{FP}}\mathbf{W}_{\text{Head}} = \begin{bmatrix} 0.5 \\ 0.44 \end{bmatrix}^T \text{ and } \mathbf{X}\mathbf{W}_q\mathbf{W}_{\text{Head}} = \begin{bmatrix} 0.5 \\ 0.51 \end{bmatrix}^T.$$

This result implies that the FP model predicts token t_1 , while its quantized counterpart instead predicts the opposite token, t_2 . Consequently, even if the final block is quantized to minimize the MSE between $\mathbf{XW}_{\text{FP}}$ and $\mathbf{XW}_q$, ignoring the LM head during block-wise quantization can lead the quantized model to produce different token predictions from the FP model.

Even when the LM head is considered, minimizing the MSE between the logits of the FP and quantized models does not guarantee identical token predictions. Suppose we obtain

$$\mathbf{XW}_q \mathbf{W}_{\text{Head}} = \begin{cases} \text{(i)} \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}^T & \text{for } \mathbf{XW}_{\text{FP}} \mathbf{W}_{\text{Head}} = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}^T \\ \text{(ii)} \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}^T & \text{for } \mathbf{XW}_{\text{FP}} \mathbf{W}_{\text{Head}} = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}^T \end{cases} \quad (1)$$

Then, the corresponding MSE values are given by

$$\begin{aligned} & \|\mathbf{XW}_{\text{FP}} \mathbf{W}_{\text{Head}} - \mathbf{XW}_q \mathbf{W}_{\text{Head}}\|_F^2 \\ &= \begin{cases} \text{(i)} (0.6 - 0.4)^2 + (0.4 - 0.6)^2 = 0.08 \\ \text{(ii)} (0.9 - 0.6)^2 + (0.1 - 0.4)^2 = 0.18 \end{cases} \end{aligned} \quad (2)$$

Although the first case (i) yields the smaller MSE, it leads to the opposite token prediction, whereas the second (ii)—despite having the larger MSE—produces the same token prediction as the FP model. This therefore demonstrates that minimizing MSE at the logit level does not necessarily align with reproducing the FP model’s token predictions. Consistently, Figure 1 (a) illustrates that standard block-wise PTQ achieves a lower MSE yet predicts a different top-1 token than the FP model, leading to an incorrect answer.

3. Method

As discussed in Section 2, ensuring that low-bit quantized LLMs reproduce the token predictions of their FP counterparts requires explicitly accounting for the LM head and replacing MSE in the optimization objective of block-wise PTQ. To this end, we propose *Logit-aware Final-block Quantization (LFQ)*, which quantizes the final Transformer block by minimizing the cross-entropy loss between the logits of the FP model and those of its quantized counterpart.

3.1. Logit-aware Final-block Quantization (LFQ)

Even when the LM head is taken into account, minimizing the MSE does not guarantee that the quantized model will predict the same token as the FP model. Since minimizing cross-entropy is equivalent to minimizing KL divergence, and KL divergence is equal to zero if and only if two distributions are identical, minimizing cross-entropy at the logit

level directly encourages the quantized model’s token-level distribution to match its FP counterpart. Furthermore, as Bruch (2021) demonstrates, cross-entropy can be used for learning to rank, which would also help the quantized model recover the FP model’s top-k token ordering. Accordingly, when optimizing the quantized final Transformer block $\mathbf{W}_q$, we minimize the cross-entropy between the FP model’s logits and those of the quantized model to align the block-wise PTQ objective with the FP model’s token generation. Specifically, while the first through penultimate Transformer blocks are quantized by minimizing the MSE between the outputs of the FP and quantized blocks, the final block is quantized using the following optimization objective:

$$\begin{aligned} & \min_{\mathbf{W}_q} \mathcal{L}_{\text{CE}}(\mathbf{XW}_{\text{FP}} \mathbf{W}_{\text{Head}}, \mathbf{XW}_q \mathbf{W}_{\text{Head}}) \\ &= -\frac{1}{L} \sum_{i,j=1}^{L,V} \sigma(\mathbf{XW}_{\text{FP}} \mathbf{W}_{\text{Head}})_{i,j} \log(\sigma(\mathbf{XW}_q \mathbf{W}_{\text{Head}})_{i,j}), \end{aligned} \quad (3)$$

where $\sigma(\mathbf{Z}) := \frac{\exp(Z_{i,j})}{\sum_{k=1}^V \exp(Z_{i,k})}$ for $\mathbf{Z} = [Z_{i,j}]_{i=1,j=1}^{L,V}$. We refer to Eq. 3 as “LFQ.”

The specific quantization parameters contained in $\mathbf{W}_q$ depend on the chosen block-wise PTQ method. For example, when instantiating (a) FlexRound (Lee et al., 2023), (b) OmniQuant (Shao et al., 2024), or (c) Block-AP (Chen et al., 2025), Eq. 3 specializes accordingly as:

$$\text{(a) } \mathbf{W}_q = \mathbf{s}_1 \left\lfloor \frac{\mathbf{W}_{\text{FP}}}{\mathbf{s}_1 \odot \mathbf{S}_2 \odot \mathbf{s}_3} \right\rfloor \quad (4)$$

$$\begin{aligned} & \text{where } \mathbf{s}_1, \mathbf{s}_3 \in \mathbb{R}_{>0}^{c_{\text{out}} \times \frac{c_{\text{in}}}{g}}, \text{ and } \mathbf{S}_2 \in \mathbb{R}_{>0}^{c_{\text{out}} \times c_{\text{in}}}, \\ & \Rightarrow \text{Eq. 3: } \min_{\mathbf{s}_1, \mathbf{S}_2, \mathbf{s}_3} \mathcal{L}_{\text{CE}}(\mathbf{XW}_{\text{FP}} \mathbf{W}_{\text{Head}}, \mathbf{XW}_q \mathbf{W}_{\text{Head}}). \end{aligned}$$

$$\begin{aligned} & \text{(b) } \mathbf{W}_q = \mathbf{h} \left\lfloor \frac{\mathbf{W}_{\text{FP}}}{\mathbf{h}} \right\rfloor \text{ where } \gamma, \beta \in \mathbb{R}_{[0,1]}^{c_{\text{out}} \times \frac{c_{\text{in}}}{g}}, \\ & \text{and } \mathbf{h} = \frac{\gamma \max(\mathbf{W}_{\text{FP}}) - \beta \min(\mathbf{W}_{\text{FP}})}{2^b - 1} \end{aligned} \quad (5)$$

$$\Rightarrow \text{Eq. 3: } \min_{\gamma, \beta} \mathcal{L}_{\text{CE}}(\mathbf{XW}_{\text{FP}} \mathbf{W}_{\text{Head}}, \mathbf{XW}_q \mathbf{W}_{\text{Head}}).$$

$$\begin{aligned} & \text{(c) } \mathbf{W}_q = \mathbf{s} \left\lfloor \frac{\mathbf{W}_{\text{FP}}}{\mathbf{s}} \right\rfloor \text{ where } \mathbf{s} \in \mathbb{R}_{>0}^{c_{\text{out}} \times \frac{c_{\text{in}}}{g}}, \\ & \Rightarrow \text{Eq. 3: } \min_{\mathbf{s}, \mathbf{W}_{\text{FP}}} \mathcal{L}_{\text{CE}}(\mathbf{XW}_{\text{FP}} \mathbf{W}_{\text{Head}}, \mathbf{XW}_q \mathbf{W}_{\text{Head}}). \end{aligned} \quad (6)$$

Here, b is the bit-width and g is the group size ($g = c_{\text{in}}$ for per-channel quantization, and $g = 128$ for group-wise quantization). We refer to Eq. 4, 5, and 6 as “FlexRound+LFQ”, “OmniQuant+LFQ”, and “Block-AP+LFQ”, respectively.

Three points are worth highlighting here. First, since LFQ integrates the LM Head and cross-entropy into the loss objective of standard block-wise PTQ, it is agnostic to the underlying block-wise method and thus can be applied seamlessly. Second, as LFQ optimizes only the final Transformer

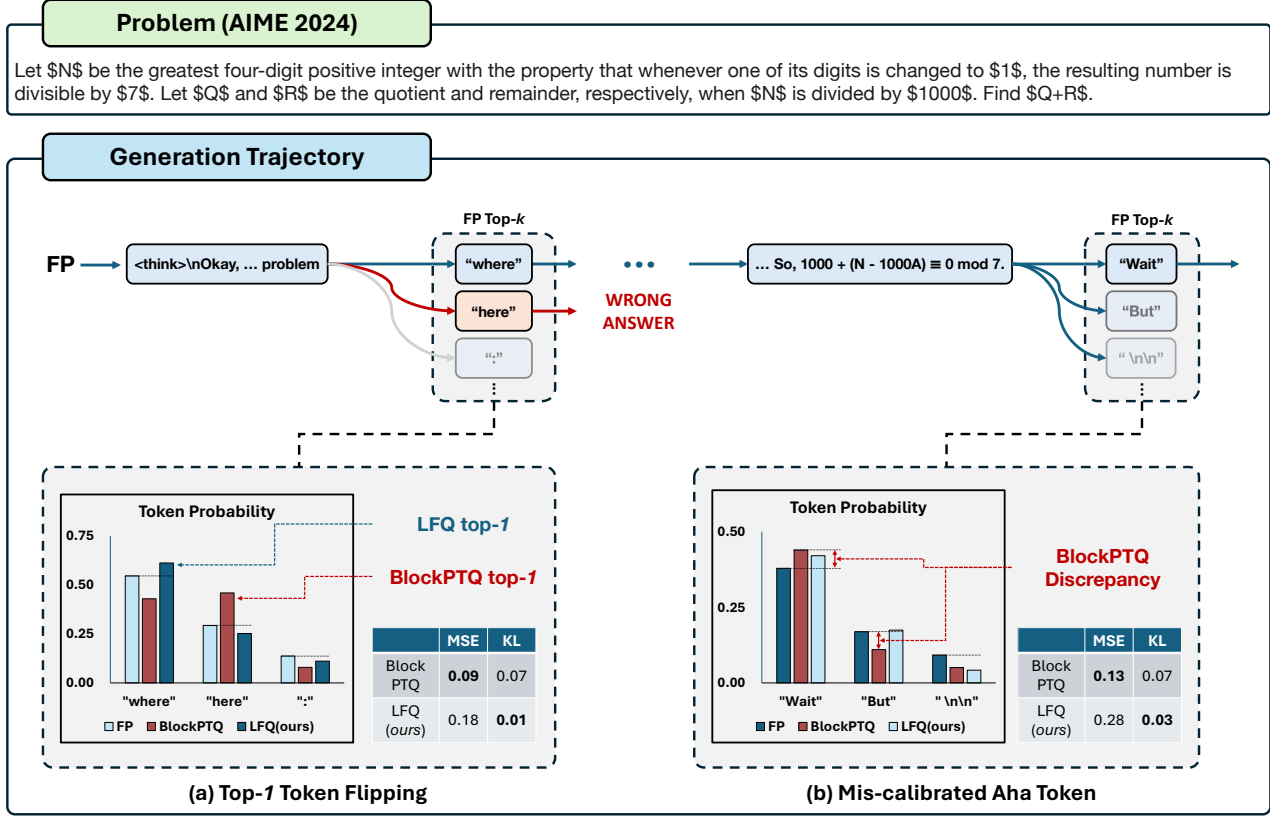


Figure 1. Reasoning trajectory of L1-Qwen-7B-Max under greedy decoding on AIME 2024 Problem 28. We compare token-level probability distributions for the FP baseline, block-wise PTQ (“blockPTQ” in this figure), and LFQ (ours) at two instants: (a) the first step where block-wise PTQ’s top-1 token diverges from the FP baseline, and (b) the first “aha” moment guiding the reasoning onto the correct path. In (a), block-wise PTQ’s top-1 (“here”) corresponds to the FP baseline’s top-2, yielding an incorrect answer, whereas LFQ’s top-1 (“where”) matches the FP baseline’s top-1 and thus reaches the correct answer. In (b), block-wise PTQ is overconfident in “Wait” and underconfident in “But”, while LFQ assigns probabilities to these “aha” tokens that remain closer to the FP baseline.

block by minimizing cross-entropy at the logit level, it is memory-efficient and thus can be run on a single GPU like other block-wise PTQ techniques. Third, because LFQ modifies only the optimization objective and leaves the quantization scheme unchanged, LFQ-quantized LLMs remain fully compatible with existing packing/unpacking kernels (e.g., Frantar et al. (2023); Lin et al. (2024); Park et al. (2024)) and can therefore be accelerated without additional effort.

3.2. Effect of LFQ on Token Generation

To illustrate that cross-entropy better reproduces the FP model’s token predictions than MSE, we revisit the example 1 in Section 2. The corresponding cross-entropy values are given as follows:

$$\begin{aligned}
 & -\frac{1}{L} \sum_{i,j=1}^{L,V} \sigma(\mathbf{X}\mathbf{W}_{\text{FP}}\mathbf{W}_{\text{Head}})_{i,j} \log(\sigma(\mathbf{X}\mathbf{W}_q\mathbf{W}_{\text{Head}}))_{i,j} \\
 & = \begin{cases} \text{(i)} & -0.6 \log(0.4) - 0.4 \log(0.6) \approx 0.75 \\ \text{(ii)} & -0.9 \log(0.6) - 0.1 \log(0.4) \approx 0.55 \end{cases}
 \end{aligned}$$

In contrast to MSE, which is lower in case (i) than in case (ii) despite case (i) predicting the opposite token and case (ii) predicting the same token as the FP model, cross-entropy assigns a smaller value to case (ii) than to case (i). This observation highlights that minimizing the cross-entropy loss at the logit level is essential for guiding low-bit quantized LLMs to align with the FP model’s token predictions.

To make this trend concrete, Figure 1 presents a reasoning trajectory generated by L1-Qwen-7B-Max for Problem 28 of AIME 2024 and compares token-level probability distributions for the FP baseline, block-wise PTQ, and LFQ at two key points: (a) the first instance where block-wise PTQ’s top-1 token diverges from the FP baseline, and (b) the first “aha” moment that steers the reasoning onto the correct path. In Figure 1 (a), although block-wise PTQ attains a smaller MSE than LFQ, thanks to minimizing cross-entropy at the logit level, LFQ yields a smaller KL divergence from the FP distribution. Consequently, LFQ reproduces the FP model’s top-1 token prediction (i.e., “where”) and follows the correct trajectory to the right answer, whereas block-wise PTQ diverges and thus fails to solve the problem.

Table 1. Performance of Qwen2.5-7B-Instruct and Qwen2.5-14B-Instruct with LFQ under block-wise PTQ (FlexRound, OmniQuant, and Block-AP). Within each PTQ method, the best accuracy is shown in **bold**. “W4” and “W3g128” denote 4-bit per-channel weight-only quantization and 3-bit group-wise quantization (group size 128), respectively. LFQ yields consistent gains in generation quality across block-wise PTQ, while preserving the language modeling and understanding performance of existing methods.

Method	# Bits	Language Modeling/Understanding		Text Generation	
		WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	MATH500 ($\uparrow$) (greedy)
Qwen2.5-7B-Instruct	BF16	6.85	73.49	70.79	74.2
FlexRound	W4	7.23	72.50	69.50	72.6
FlexRound+LFQ (Ours)	W4	7.21	72.48	71.35	73.4
FlexRound	W3g128	7.63	70.13	66.54	65.6
FlexRound+LFQ (Ours)	W3g128	7.58	70.26	67.84	68.0
OmniQuant	W4	7.73	71.00	68.21	69.8
OmniQuant+LFQ (Ours)	W4	7.53	70.99	69.50	71.6
OmniQuant	W3g128	8.08	68.43	68.21	63.6
OmniQuant+LFQ (Ours)	W3g128	7.91	68.39	68.58	64.4
Block-AP	W4	7.87	69.60	66.73	68.0
Block-AP+LFQ (Ours)	W4	7.77	69.94	68.02	69.0
Block-AP	W3g128	8.70	67.09	61.00	60.0
Block-AP+LFQ (Ours)	W3g128	8.18	67.06	63.77	61.8
Qwen2.5-14B-Instruct	BF16	5.24	78.82	79.85	78.4
FlexRound	W4	5.67	77.33	77.82	76.4
FlexRound+LFQ (Ours)	W4	5.62	77.31	78.00	77.2
FlexRound	W3g128	6.15	75.84	75.05	69.6
FlexRound+LFQ (Ours)	W3g128	6.11	75.85	77.08	71.6
OmniQuant	W4	5.93	76.64	73.94	73.4
OmniQuant+LFQ (Ours)	W4	5.89	76.66	75.23	75.2
OmniQuant	W3g128	6.43	75.62	74.31	70.4
OmniQuant+LFQ (Ours)	W3g128	6.36	75.73	75.42	69.8
Block-AP	W4	6.23	76.84	70.79	71.6
Block-AP+LFQ (Ours)	W4	6.17	76.86	72.27	72.4
Block-AP	W3g128	6.81	74.58	71.72	67.0
Block-AP+LFQ (Ours)	W3g128	6.69	74.58	72.46	68.0

Moreover, because the occurrence of an “aha” moment is pivotal for re-evaluating and correcting an ongoing reasoning trajectory, the extent to which low-bit quantized models track the FP baseline on such “aha” tokens—e.g., “Wait” and “But”—is a key determinant of their accuracy on complex reasoning benchmarks. As shown in Figure 1 (b), even when the top-1 token for all three models is “Wait”, it is noteworthy that block-wise PTQ is overconfident in “Wait”, leaving it underconfident in another “aha” token like “But”. By contrast, LFQ allocates these probabilities closer to the FP baseline, resulting in not only a smaller KL divergence but also higher accuracy than block-wise PTQ as reported in Table 2.

4. Experiment

In this section, we first assess LFQ on Qwen2.5-7B-Instruct and Qwen2.5-14B-Instruct (Qwen et al., 2025) using IFEval and MATH500. We then evaluate LFQ on reasoning

models—L1-Qwen-7B-Max (Aggarwal & Welleck, 2025) and DeepSeek-R1-Distill-Llama-8B (DeepSeek-AI et al., 2025)—on MATH500 and AIME 2024 (AIME’24). We also validate LFQ on Qwen3-30B-A3B-Instruct-2507 (Team, 2025) using IFEval and AIME 2025 (AIME’25). Finally, we empirically (i) demonstrate the importance of incorporating the LM head and utilizing cross-entropy in the objective, (ii) validate that quantizing only the final Transformer block via logit-level cross-entropy is sufficient (i.e., it is not a must to quantize multiple final blocks with cross-entropy), and (iii) the comparison of LFQ against LoRA-based quantization error compensation (LQEC). These findings are mainly established on Llama 3.1 8B Instruct (Grattafiori et al., 2024) using IFEval and GSM8K.

We select calibration samples from the beginning of C4 training set (Raffel et al., 2020) in sequential order. We do so to emphasize that LFQ can preserve performance comparable to FP baselines on language modeling (e.g., WikiText-2) and understanding (e.g., MMLU), while consis-

Table 2. Performance of L1-Max-Qwen-7B and DeepSeek-R1-Distill-Llama-8B with LFQ under block-wise PTQ (FlexRound). Within the PTQ method, the best accuracy is shown in **bold**. “W4” and “W3g128” denote 4-bit per-channel weight-only quantization and 3-bit group-wise quantization (group size 128), respectively. LFQ yields consistent gains in generation quality, while preserving the language modeling and understanding performance of existing methods. To estimate avg@8 and pass@8, we use temperature 0.6 and top-p 0.95 with a maximum generation length of 4096 tokens. For pass@8, we sample 16 responses per question.

Method	# Bits	Language Modeling/Understanding		Text Generation		
		WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	MATH500 ($\uparrow$) (avg@8)	AIME’24 ($\uparrow$) (greedy)	AIME’24 ($\uparrow$) (pass@8)
L1-Qwen-7B-Max	BF16	29.57	54.58	89.05 $\pm$ 0.74	46.67	55.30
FlexRound	W4	31.20	53.43	87.45 $\pm$ 0.75	30.00	51.71
FlexRound+LFQ (Ours)	W4	30.44	53.10	88.40 $\pm$ 0.86	43.33	55.09
FlexRound	W3g128	31.45	52.24	85.35 $\pm$ 0.64	23.33	41.85
FlexRound+LFQ (Ours)	W3g128	29.46	52.53	86.50 $\pm$ 0.54	30.00	45.18
DeepSeek-R1-Distill-Llama-8B	BF16	11.85	55.69	72.53 $\pm$ 1.16	30.00	30.49
FlexRound	W4	12.61	54.57	70.10 $\pm$ 1.37	16.67	27.71
FlexRound+LFQ (Ours)	W4	12.46	54.21	71.95 $\pm$ 1.20	26.67	30.07
FlexRound	W3g128	13.80	53.61	67.00 $\pm$ 0.97	10.00	15.98
FlexRound+LFQ (Ours)	W3g128	13.24	54.03	69.25 $\pm$ 0.85	13.33	16.97

Table 3. Performance of Qwen3-30B-A3B-Instruct-2507 with LFQ under block-wise PTQ (FlexRound). Within the PTQ method, the best accuracy is shown in **bold**. “W4g128” denote 4-bit group-wise quantization (group size 128). LFQ yields consistent gains in generation quality, while preserving the language modeling and understanding performance of existing methods. To estimate pass@1, we use temperature 0.7, top-p 0.8, and top-k 20, and sample 16 responses per question with a maximum generation length of 32768 tokens.

Method	# Bits	Language Modeling/Understanding		Text Generation		
		WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	AIME’25 ($\uparrow$) (greedy)	AIME’25 ($\uparrow$) (pass@1)
Qwen3-30B-A3B-Instruct-2507	BF16	7.00	81.12	83.18	66.67	62.50
GPTQ	W4g128	7.32	80.15	82.26	50.00	55.63
FlexRound	W4g128	7.33	80.24	82.44	53.33	57.29
FlexRound+LFQ (Ours)	W4g128	7.27	80.25	82.99	60.00	58.75

tently improving the generation quality of block-wise PTQ. We report perplexity on WikiText2 using a sequence length of 4096, five-shot accuracy on MMLU, prompt-level strict-accuracy on IFEval (following Qwen et al. (2025)), 8-shot accuracy on GSM8K, and zero-shot accuracy on MATH500 and AIME. Further setup details are given in Appendix E.

4.1. Qwen2.5-Instruct on IFEval and MATH500

To verify whether LFQ can improve low-bit instruction-tuned LLMs on both natural language instruction following and challenging math word problems, we evaluate LFQ for Qwen2.5-7B-Instruct and Qwen2.5-14B-Instruct on IFEval and MATH500 using greedy decoding. Table 1 shows that, across different quantization configurations, LFQ consistently improves the generation quality of instruction-tuned models quantized by FlexRound, OmniQuant, and Block-AP on both IFEval and MATH500. Consequently, FlexRound+LFQ narrows the gap between 4-bit per-channel models and their FP baselines to within 1 percentage point

(pp) for Qwen2.5-7B-Instruct and within 2 pp for Qwen2.5-14B-Instruct across all benchmarks considered.

4.2. Reasoning Models on MATH500 and AIME’24

To test whether LFQ can also perform well for reasoning models that produce long chains of thought by scaling test-time compute, we apply LFQ to L1-Qwen-7B-Max and DeepSeek-R1-Distill-Llama-8B on MATH500 and AIME’24. Given that Table 1 identifies FlexRound+LFQ as the most effective, we focus on FlexRound+LFQ hereafter, unless otherwise noted. Table 2 shows that block-wise PTQ suffers substantial degradation under greedy decoding on AIME’24. In contrast, LFQ nearly matches the FP baseline on AIME’24, indicating that it restores alignment with the FP model’s top-1 token predictions. Furthermore, LFQ raises pass@8 to within 0.5 percentage points of the FP baselines. Taken together, these results suggest that LFQ effectively aligns the token-level probabilities of low-bit quantized LLMs with those of their FP counterparts.

Table 4. Performance of Llama 3.1 8B Instruct when block-wise PTQ methods (FlexRound, OmniQuant, and Block-AP) are incrementally augmented by (i) incorporating the LM head and (ii) using a logit-level cross-entropy objective in order to quantize the final Transformer block. Within each block-wise PTQ method, the best accuracy is shown in **bold** and the second-best is underlined. Here, all results use 4-bit per-channel weight-only quantization. LFQ (with both LM Head and cross-entropy, ours) yields consistent gains in generation quality across block-wise PTQ, while preserving the language modeling and understanding performance of existing methods.

Method	Language Modeling/Understanding				Text Generation	
	LM-Head	Cross-Entropy	WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	GSM8K ($\uparrow$) (greedy)
Llama 3.1 8B Instruct	N/A	N/A	6.75	68.34	74.49	84.99
FlexRound	X	X	<u>7.06</u>	66.19	70.24	81.35
FlexRound+LFQ	O	X	<u>7.08</u>	<u>66.75</u>	<u>71.53</u>	<u>81.58</u>
FlexRound+LFQ	O	O	7.06	66.97	72.09	81.80
OmniQuant	X	X	7.49	<u>64.87</u>	70.61	78.17
OmniQuant+LFQ	O	X	<u>7.48</u>	<u>64.77</u>	<u>71.35</u>	<u>78.32</u>
OmniQuant+LFQ	O	O	7.47	65.48	71.35	79.76
Block-AP	X	X	7.76	63.24	68.58	73.84
Block-AP+LFQ	O	X	<u>7.74</u>	<u>63.54</u>	<u>68.39</u>	<u>74.00</u>
Block-AP+LFQ	O	O	7.69	63.77	68.76	74.45

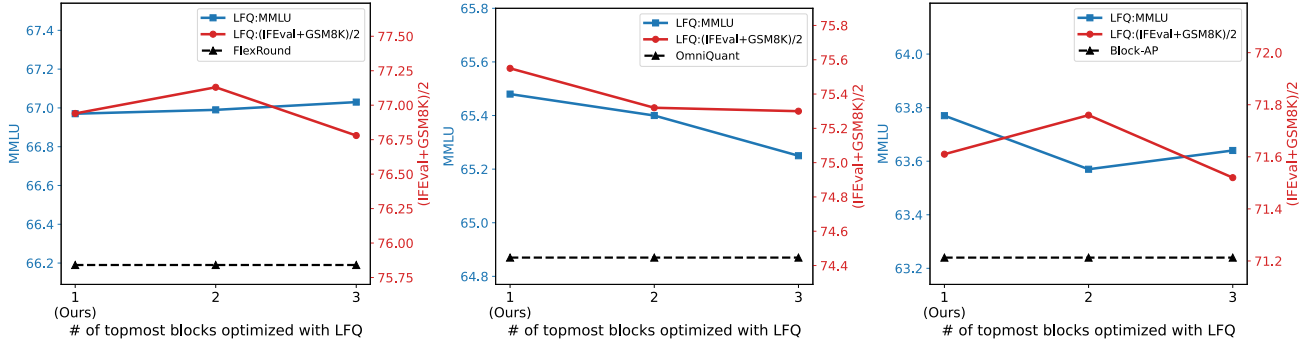


Figure 2. Performance of Llama 3.1 8B Instruct as the number of topmost Transformer blocks optimized with LFQ increases from 1 (ours) to 3, with the remaining blocks quantized via standard MSE reconstruction; shown for FlexRound (left), OmniQuant (center), and Block-AP (right). In each subfigure, the left y-axis shows MMLU accuracy, while the right y-axis reports the IFEval+GSM8K average. All results use 4-bit per-channel weight-only quantization. The average of IFEval and GSM8K (expressed as “(IFEval+GSM8K)/2”) stays roughly unchanged, regardless of the number of topmost Transformer blocks optimized with LFQ.

4.3. Qwen3-30B-A3B-Instruct-2507 on IFEval and AIME/25

We conduct additional LFQ experiments on Qwen3-30B-A3B-Instruct-2507 to demonstrate that LFQ is effective for both MoE and traditional dense architectures. As shown in Table 3, LFQ improves the pass@1 score on AIME/25 by approximately 1.5 pp and the IFEval score by about 0.5 pp, thereby narrowing the performance gap between the quantized MoE model and its FP counterpart.

4.4. Ablation Study

Importance of LM Head and cross-entropy. To assess the impact of incorporating the LM head and using cross-entropy in the loss objective when quantizing the final block, we incrementally augment existing block-wise PTQ methods (FlexRound, OmniQuant, and Block-AP) with these

components. As shown in Table 4, adding the LM head alone generally improves accuracy on text generation benchmarks (IFEval and GSM8K) as well as on language modeling and understanding. With the LM head in place, employing cross-entropy rather than mean squared error (MSE) yields further gains on text generation tasks. We therefore conclude that leveraging both the LM head and cross-entropy, as in Eq. 3, is essential for boosting the generation quality of low-bit quantized LLMs. We conduct additional ablation study for Qwen2.5-7B-Instruct in Appendix B.

Sufficiency of quantizing solely the final block via logit-level cross-entropy. We ask whether applying the logit-level cross-entropy objective to only the final block is sufficient. To test this, we vary the number of topmost Transformer blocks optimized with LFQ (denoted as k) while keeping the remaining blocks quantized via standard MSE reconstruction. For example, when $k = 2$, we apply LFQ se-

Table 5. Comparison of LFQ (ours) against RILQ, a state-of-the-art LoRA-based quantization error compensation method, on Llama 3.1 8B Instruct using block-wise PTQ (FlexRound, OmniQuant, and Block-AP). Within each PTQ method, the best accuracy is shown in **bold** and the second best is underlined. “W4” denotes 4-bit per-channel weight-only quantization.

Method	# Bits	Language modeling/understanding		Text generation	
		WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	GSM8K ($\uparrow$) (greedy)
Llama 3.1 8B Instruct	BF16	6.75	68.34	74.49	84.99
FlexRound	W4	7.06	66.19	70.24	81.35
FlexRound+RILQ	W4	6.95	<u>66.86</u>	<u>71.90</u>	80.52
FlexRound+LFQ	W4	<u>7.06</u>	66.97	72.09	81.80
OmniQuant	W4	7.49	64.87	70.61	78.17
OmniQuant+RILQ	W4	7.24	66.07	<u>71.35</u>	<u>78.85</u>
OmniQuant+LFQ	W4	<u>7.47</u>	<u>65.48</u>	71.35	79.76
Block-AP	W4	7.76	63.24	68.58	73.84
Block-AP+RILQ	W4	7.43	64.62	<u>68.58</u>	<u>73.92</u>
Block-AP+LFQ	W4	<u>7.69</u>	<u>63.77</u>	68.76	74.45

quentially to the penultimate and final blocks. As shown in Figure 2, the average score of IFEval and GSM8K remains almost constant even as k increases; $k = 2$ occasionally yields a marginal gain on that average but at the cost of lower MMLU accuracy. These results indicate that applying LFQ to the final block alone is sufficient and offers the best overall trade-off. We also emphasize in Appendix C that *to which* block LFQ is applied is far more important than *how many* blocks are optimized with LFQ.

Comparison of LFQ against LQEC. As LQEC has emerged as a promising approach for mitigating memory bottleneck while recovering task accuracy, we compare LFQ with RILQ (Lee et al., 2025a), a state-of-the-art LQEC method. For a fair comparison, we use the C4 training set as calibration data to initialize LoRA adapters on low-bit quantized models produced by FlexRound, OmniQuant, and Block-AP. In Table 5, RILQ often outperforms LFQ on language modeling (WikiText2) and understanding (MMLU) due to its use of LoRA adapters, but LFQ consistently surpasses RILQ on text generation (IFEval and GSM8K) across all settings. We hypothesize that this stems from the fact that LQEC methods—including RILQ—optimize MSE, an objective misaligned with matching the FP model’s token-level distribution (as elucidated in Section 2). To leverage the strengths of both methods, we also explore their joint application in Appendix D.

5. Related Work

It is well known that quantization-aware training (QAT) can match full-precision (FP) accuracy even under sub-4-bit quantization configurations, so it has been applied across domains—from computer vision models (Esser et al., 2020; Lee et al., 2021) to natural language models (Liu et al., 2023;

2025). However, Liu et al. (2025) shows that QAT requires fine-tuning LLMs on billions of tokens at least, which is prohibitively memory-intensive and time-consuming. Consequently, research attention has continued to focus more on advancing post-training quantization (PTQ).

PTQ is commonly divided into layer-wise and block-wise methods. Layer-wise PTQ (e.g., Frantar et al. (2023); Lin et al. (2024)) can be run fast on a single GPU and typically incurs marginal performance degradation on relatively easy downstream tasks (e.g., commonsense reasoning). However, as these techniques do not involve gradient-based optimization, unless task-specific calibration data is utilized, they can suffer substantial accuracy degradation on more challenging benchmarks, particularly text generation (Li et al., 2025). On the other hand, block-wise PTQ approaches (Lee et al., 2023; Shao et al., 2024; Cheng et al., 2024; Lee et al., 2025b; Chen et al., 2025; Park et al., 2025) not only account for cross-layer dependencies within a block but also optimize quantization parameters via gradient-based iterative updates, and therefore often outperform layer-wise PTQ. Notwithstanding, we find that existing block-wise PTQ can still exhibit non-negligible degradation in generation quality.

6. Conclusion

We show that block-wise PTQ can degrade generation quality due to (i) omitting the LM head from block-wise optimization and (ii) relying on the MSE objective. To address this, we introduce Logit-aware Final-block Quantization (LFQ), which quantizes the final Transformer block by aligning the quantized model’s logits to the FP model’s via the cross-entropy loss. Across diverse model families and generation tasks, LFQ consistently improves generation quality over existing block-wise PTQ techniques, while preserving performance on language modeling and understanding.

Acknowledgements

This work was supported by Institute for Information & communications Technology Planning & Evaluation(IITP)grant funded by the Korea government(MSIT) (RS-2019-II190075, Artificial Intelligence Graduate School Program(KAIST)) and National Research Foundation of Korea (NRF) grant (No.RS-2023-00209060, A Study on Optimization and Network Interpretation Method for Large-Scale Machine Learning) funded by the Korea government (MSIT). This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2026-25507427, Development of Efficient Architectures and Training Techniques for High-Performance Lightweight AI Models).

Impact Statement

This paper presents work aimed at improving the inference efficiency of large language models. We anticipate several potential societal implications, but we do not believe any require specific emphasis here.

References

- Aggarwal, P. and Welleck, S. L1: Controlling how long a reasoning model thinks with reinforcement learning, 2025. URL <https://arxiv.org/abs/2503.04697>.
- AIME. Aime problems and solutions, 2025. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.
- Bruch, S. An alternative cross entropy loss for learning-to-rank. In *Proceedings of the Web Conference 2021*, WWW ’21, pp. 118–126, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383127. doi: 10.1145/3442381.3449794. URL <https://doi.org/10.1145/3442381.3449794>.
- Chen, M., Shao, W., Xu, P., Wang, J., Gao, P., Zhang, K., and Luo, P. EfficientQAT: Efficient quantization-aware training for large language models. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10081–10100, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.498. URL <https://aclanthology.org/2025.acl-long.498/>.
- Cheng, W., Zhang, W., Shen, H., Cai, Y., He, X., Kaokao, L., and Liu, Y. Optimize weight rounding via signed gradient descent for the quantization of LLMs. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 11332–11350, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.662. URL <https://aclanthology.org/2024.findings-emnlp.662/>.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Ding, H., Xin, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Wang, J., Chen, J., Yuan, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Ye, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Zhao, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Esser, S. K., McKinstry, J. L., Bablani, D., Appuswamy, R., and Modha, D. S. Learned step size quantization, 2020. URL <https://arxiv.org/abs/1902.08153>.
- Frantar, E., Ashkboos, S., Hoeffler, T., and Alistarh, D. Gptq:

Accurate post-training quantization for generative pre-trained transformers, 2023. URL <https://arxiv.org/abs/2210.17323>.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lekomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhotia, K., Rantala-Yeary, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Celebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhennde, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z.,

Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damaj, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A. L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S.,

- Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Lee, G., Lee, J., Hong, S., Kim, M., Ahn, E., Chang, D.-S., and Choi, J. Rilq: Rank-insensitive lora-based quantization error compensation for boosting 2-bit large language model accuracy, 2025a. URL <https://arxiv.org/abs/2412.01129>.
- Lee, J. H., Yun, J., Hwang, S. J., and Yang, E. Cluster-promoting quantization with bit-drop for minimizing network quantization loss, 2021. URL <https://arxiv.org/abs/2109.02100>.
- Lee, J. H., Kim, J., Kwon, S. J., and Lee, D. FlexRound: Learnable rounding based on element-wise division for post-training quantization. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 18913–18939. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/lee23h.html>.
- Lee, J. H., Kim, J., Yang, J. Y., Kwon, S. J., Yang, E., Yoo, K. M., and Lee, D. LRQ: Optimizing post-training quantization for large language models by learning low-rank weight-scaling matrices. In Chiruzzo, L., Ritter, A., and Wang, L. (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7708–7743, Albuquerque, New Mexico, April 2025b. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.393. URL <https://aclanthology.org/2025.naacl-long.393/>.
- Li, Z., Su, Y., Yang, R., Xie, C., Wang, Z., Xie, Z., Wong, N., and Yang, H. Quantization meets reasoning: Exploring llm low-bit quantization degradation for mathematical reasoning, 2025. URL <https://arxiv.org/abs/2501.03035>.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for llm compression and acceleration, 2024. URL <https://arxiv.org/abs/2306.00978>.
- Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., Shi, Y., Krishnamoorthi, R., and Chandra, V. Llm-qat: Data-free quantization aware training for large language models, 2023. URL <https://arxiv.org/abs/2305.17888>.
- Liu, Z., Zhao, C., Huang, H., Chen, S., Zhang, J., Zhao, J., Roy, S., Jin, L., Xiong, Y., Shi, Y., Xiao, L., Tian, Y., Soran, B., Krishnamoorthi, R., Blankevoort, T., and Chandra, V. Paretoq: Scaling laws in extremely low-bit llm quantization, 2025. URL <https://arxiv.org/abs/2502.02631>.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models, 2016.
- Park, G., Park, B., Kim, M., Lee, S., Kim, J., Kwon, B., Kwon, S. J., Kim, B., Lee, Y., and Lee, D. Lut-gemm: Quantized matrix multiplication based on luts for efficient inference in large-scale generative language models. In *ICLR*, 2024. URL <https://openreview.net/forum?id=gLARhFLE0F>.
- Park, S., Bae, J., Kwon, B., Kim, M., Kim, B., Kwon, S. J., Kang, U., and Lee, D. Unifying uniform and binary-coding quantization for accurate compression of large language models. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28468–28488, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1382. URL <https://aclanthology.org/2025.acl-long.1382/>.

- Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.
- Shao, W., Chen, M., Zhang, Z., Xu, P., Zhao, L., Li, Z., Zhang, K., Gao, P., Qiao, Y., and Luo, P. Omniquant: Omnidirectionally calibrated quantization for large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=8Wuvhh0LYW>.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., Chen, Z., Cui, J., Ding, H., Dong, M., Du, A., Du, C., Du, D., Du, Y., Fan, Y., Feng, Y., Fu, K., Gao, B., Gao, H., Gao, P., Gao, T., Gu, X., Guan, L., Guo, H., Guo, J., Hu, H., Hao, X., He, T., He, W., He, W., Hong, C., Hu, Y., Hu, Z., Huang, W., Huang, Z., Huang, Z., Jiang, T., Jiang, Z., Jin, X., Kang, Y., Lai, G., Li, C., Li, F., Li, H., Li, M., Li, W., Li, Y., Li, Y., Li, Z., Li, Z., Lin, H., Lin, X., Lin, Z., Liu, C., Liu, C., Liu, H., Liu, J., Liu, J., Liu, L., Liu, S., Liu, T. Y., Liu, T., Liu, W., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Z., Lu, E., Lu, L., Ma, S., Ma, X., Ma, Y., Mao, S., Mei, J., Men, X., Miao, Y., Pan, S., Peng, Y., Qin, R., Qu, B., Shang, Z., Shi, L., Shi, S., Song, F., Su, J., Su, Z., Sun, X., Sung, F., Tang, H., Tao, J., Teng, Q., Wang, C., Wang, D., Wang, F., Wang, H., Wang, J., Wang, J., Wang, J., Wang, S., Wang, S., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Z., Wang, Z., Wang, Z., Wei, C., Wei, Q., Wu, W., Wu, X., Wu, Y., Xiao, C., Xie, X., Xiong, W., Xu, B., Xu, J., Xu, J., Xu, L. H., Xu, L., Xu, S., Xu, W., Xu, X., Xu, Y., Xu, Z., Yan, J., Yan, Y., Yang, X., Yang, Y., Yang, Z., Yang, Z., Yang, Z., Yao, H., Yao, X., Ye, W., Ye, Z., Yin, B., Yu, L., Yuan, E., Yuan, H., Yuan, M., Zhan, H., Zhang, D., Zhang, H., Zhang, W., Zhang, X., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Z., Zhao, H., Zhao, Y., Zheng, H., Zheng, S., Zhou, J., Zhou, X., Zhou, Z., Zhu, Z., Zhuang, W., and Zu, X. Kimi k2: Open agentic intelligence, 2025. URL <https://arxiv.org/abs/2507.20534>.
- Team, Q. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. Instruction-following evaluation for large language models, 2023. URL <https://arxiv.org/abs/2311.07911>.

A. Additional Avg@8 Results on IFEval and MATH500 for Qwen2.5-7B-Instruct

Table 6. Performance of Qwen2.5-7B-Instruct with LFQ under block-wise PTQ (FlexRound, OmniQuant, and Block-AP). Within each PTQ method, the best accuracy is shown in **bold**. “W4” and “W3g128” denote 4-bit per-channel weight-only quantization and 3-bit group-wise quantization (group size 128), respectively. To estimate avg@8, we use temperature 0.7, top-p 0.8, and top-k 20.

Method	# Bits	Language Modeling/Understanding		Text Generation	
		WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (avg@8)	MATH500 ($\uparrow$) (avg@8)
Qwen2.5-7B-Instruct	BF16	6.85	73.49	70.82 $\pm$ 0.88	73.65 $\pm$ 1.09
FlexRound	W4	7.23	72.50	69.64 $\pm$ 0.65	69.33 $\pm$ 1.21
FlexRound+LFQ (Ours)	W4	7.21	72.48	71.44 $\pm$ 0.55	70.35 $\pm$ 1.16
FlexRound	W3g128	7.63	70.13	68.30 $\pm$ 1.07	63.83 $\pm$ 0.92
FlexRound+LFQ (Ours)	W3g128	7.58	70.26	69.50 $\pm$ 0.56	65.35 $\pm$ 0.76
OmniQuant	W4	7.73	71.00	68.46 $\pm$ 0.72	68.28 $\pm$ 1.00
OmniQuant+LFQ (Ours)	W4	7.53	70.99	69.78 $\pm$ 0.91	69.53 $\pm$ 1.00
OmniQuant	W3g128	8.08	68.43	68.23 $\pm$ 1.04	61.75 $\pm$ 1.12
OmniQuant+LFQ (Ours)	W3g128	7.91	68.39	68.63 $\pm$ 0.45	63.58 $\pm$ 0.89
Block-AP	W4	7.87	69.60	66.87 $\pm$ 0.85	65.98 $\pm$ 1.42
Block-AP+LFQ (Ours)	W4	7.77	69.94	68.12 $\pm$ 0.97	67.25 $\pm$ 1.15
Block-AP	W3g128	8.70	67.09	61.35 $\pm$ 1.05	58.08 $\pm$ 0.82
Block-AP+LFQ (Ours)	W3g128	8.18	67.06	64.03 $\pm$ 0.79	61.53 $\pm$ 1.25

Table 6 shows that LFQ also improves Avg@8 scores on IFEval and MATH500 for Qwen2.5-7B-Instruct across different block-wise PTQ methods (FlexRound, OmniQuant, and Block-AP) and quantization schemes (W4 and W3g128), demonstrating that LFQ is effective under both greedy and stochastic decoding.

B. Additional Ablation Studies of LFQ

Table 7. Performance of Qwen2.5-7B-Instruct when block-wise PTQ methods (FlexRound, OmniQuant, and Block-AP) are incrementally augmented by (i) incorporating the LM head and (ii) using a logit-level cross-entropy objective in order to quantize the final Transformer block. Within each block-wise PTQ method, the best accuracy is shown in **bold** and the second-best is underlined. Here, all results use 4-bit per-channel weight-only quantization. LFQ (with both LM Head and cross-entropy, ours) yields consistent gains in generation quality across block-wise PTQ, while preserving the language modeling and understanding performance of existing methods.

Method	Language Modeling/Understanding				Text Generation	
	LM-Head	Cross-Entropy	WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	MATH500 ($\uparrow$) (greedy)
Qwen2.5-7B-Instruct	N/A	N/A	6.85	73.49	70.79	74.2
FlexRound	X	X	<u>7.23</u>	72.50	69.50	<u>72.6</u>
FlexRound+LFQ	O	O	7.21	<u>72.48</u>	71.35	73.4
OmniQuant	X	X	7.73	<u>71.00</u>	68.21	69.8
OmniQuant+LFQ	O	O	7.29	71.02	<u>68.95</u>	<u>70.6</u>
Block-AP	X	X	<u>7.87</u>	69.60	66.73	68.0
Block-AP+LFQ	O	O	7.77	69.94	68.02	69.0

Table 8. Performance of Llama 3.2 3B Instruct when block-wise PTQ methods (FlexRound, OmniQuant, and Block-AP) are incrementally augmented by (i) incorporating the LM head and (ii) using a logit-level cross-entropy objective in order to quantize the final Transformer block. Within each block-wise PTQ method, the best accuracy is shown in **bold** and the second-best is underlined. Here, all results use 4-bit per-channel weight-only quantization. LFQ (with both LM Head and cross-entropy, ours) yields consistent gains in generation quality across block-wise PTQ, while preserving the language modeling and understanding performance of existing methods.

Method	Language Modeling/Understanding				Text Generation	
	LM-Head	Cross-Entropy	WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	GSM8K ($\uparrow$) (greedy)
Llama 3.2 3B Instruct	N/A	N/A	10.14	61.34	71.72	77.48
FlexRound	X	X	<u>10.72</u>	59.93	65.80	72.40
FlexRound+LFQ	O	O	10.72	<u>59.84</u>	67.28	73.09
OmniQuant	X	X	11.23	57.78	64.33	71.42
OmniQuant+LFQ	O	O	11.16	58.94	66.54	71.49
Block-AP	X	X	<u>11.61</u>	56.47	62.29	66.11
Block-AP+LFQ	O	O	11.61	56.38	63.77	66.41

C. Importance of Applying LFQ to the Last Block

To emphasize that *to which* block LFQ is applied is far more important than *how many* blocks are optimized with LFQ, we conduct the following experiments for Llama 3.1 8B Instruct. When $k = 2$, we apply LFQ to the second-to-last block, while for the last block we only minimize $\|XW_{FP} - XW_q\|_F^2$ without using either the LM head or cross-entropy (denoted as “ $k = 2$ except the last block”). When $k = 3$, we apply LFQ sequentially to the third- and second-to-last blocks, and again, for the last block only, we minimize $\|XW_{FP} - XW_q\|_F^2$ without employing the LM head or cross-entropy (denoted as “ $k = 3$ except the last block”). We then compare these settings with the original $k = 2$ and $k = 3$ configurations in Figure 2.

Table 9. Comparison of $k = 2$ and $k = 3$ except the last block with the original $k = 2$ and $k = 3$ configurations in Figure 2. “LFQ@Last” indicates whether LFQ is applied to the last block. Similarly, “LFQ@Last-1” and “LFQ@Last-2” indicate whether LFQ is applied to the second-to-last and third-to-last blocks, respectively.

Method	LFQ@Last	LFQ@Last-1	LFQ@Last-2	MMLU ($\uparrow$)	(IFEval+GSM8K)/2 ($\uparrow$)
Llama 3.1 8B Instruct	N/A	N/A	N/A	68.34	79.74
FlexRound	X	X	X	66.19	75.80
+ $k = 2$ except the last block	$\bar{X}$	$\bar{O}$	$\bar{X}$	66.97	76.17
+ $k = 2$ (Figure 2)	O	O	X	66.99	77.13
+ $k = 3$ except the last block	$\bar{X}$	$\bar{O}$	$\bar{O}$	66.98	76.15
+ $k = 3$ (Figure 2)	O	O	O	67.03	76.78
OmniQuant	X	X	X	64.87	74.39
+ $k = 2$ except the last block	$\bar{X}$	$\bar{O}$	$\bar{X}$	65.29	74.47
+ $k = 2$ (Figure 2)	O	O	X	65.40	75.32
+ $k = 3$ except the last block	$\bar{X}$	$\bar{O}$	$\bar{O}$	65.29	74.65
+ $k = 3$ (Figure 2)	O	O	O	65.25	75.30
Block-AP	X	X	X	63.24	71.21
+ $k = 2$ except the last block	$\bar{X}$	$\bar{O}$	$\bar{X}$	63.78	71.30
+ $k = 2$ (Figure 2)	O	O	X	63.57	71.76
+ $k = 3$ except the last block	$\bar{X}$	$\bar{O}$	$\bar{O}$	63.81	71.24
+ $k = 3$ (Figure 2)	O	O	O	63.64	71.52

As shown in Table 9, the MMLU score remains nearly unchanged regardless of whether LFQ is applied to the final block. In contrast, when LFQ is not applied to the final block, the average of IFEval and GSM8K (i.e., “(IFEval+GSM8K)/2”) consistently drops, approaching the performance level of each underlying PTQ technique. These results indicate that, for improving the generation quality of low-bit quantized LLMs, it is far more critical to apply LFQ to the final block than to simply increase the number of blocks optimized with LFQ.

D. Combination of LFQ with RILQ

Table 10. Comparison of LFQ (ours) against RILQ, a state-of-the-art LoRA-based quantization error compensation method, on Llama 3.1 8B Instruct using block-wise PTQ (FlexRound, OmniQuant, and Block-AP). Within each PTQ method, the best accuracy is shown in **bold** and the second best is underlined. “W4” denotes 4-bit per-channel weight-only quantization.

Method	# Bits	Language modeling/understanding		Text generation	
		WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	GSM8K ($\uparrow$) (greedy)
Llama 3.1 8B Instruct	BF16	6.75	68.34	74.49	84.99
FlexRound	W4	7.06	66.19	70.24	81.35
FlexRound+RILQ	W4	6.95	66.86	71.90	80.52
FlexRound+LFQ	W4	7.06	66.97	<u>72.09</u>	81.80
FlexRound+LFQ+RILQ	W4	<u>6.98</u>	<u>66.96</u>	72.46	<u>81.43</u>
OmniQuant	W4	7.49	64.87	70.61	78.17
OmniQuant+RILQ	W4	<u>7.24</u>	66.07	<u>71.35</u>	78.85
OmniQuant+LFQ	W4	7.47	65.48	<u>71.35</u>	79.76
OmniQuant+LFQ+RILQ	W4	7.23	<u>65.82</u>	71.35	<u>79.45</u>
Block-AP	W4	7.76	63.24	68.58	73.84
Block-AP+RILQ	W4	<u>7.43</u>	64.62	68.58	73.92
Block-AP+LFQ	W4	7.69	63.77	68.76	74.45
Block-AP+LFQ+RILQ	W4	7.43	<u>64.53</u>	<u>68.58</u>	<u>74.22</u>

LFQ underperforms RILQ on WikiText2 perplexity (language modeling) and MMLU accuracy (language understanding), while outperforming RILQ on IFEval and GSM8K (text generation), as shown in Table 5. However, we emphasize that LFQ (quantization objective) and RILQ (LoRA addition) address orthogonal aspects of the problem rather than competing with each other. Because LFQ can be readily combined with RILQ in a complementary manner, we therefore explore their joint application to leverage the strengths of both methods.

LFQ + RILQ performs comparably to RILQ on WikiText2 perplexity and MMLU accuracy, while achieving results close to LFQ on IFEval and GSM8K. This indicates that LFQ + RILQ can effectively inherit the strengths of both techniques.

E. Experimental Setting of LFQ

We sweep the LFQ learning rate as follows: $\{5e-4, 1e-3\}$ with FlexRound; $\{1.5e-3, 2e-3, 5e-3\}$ with OmniQuant; and $\{1e-5, 2e-5, 3e-5\}$ with Block-AP. Across all block-wise PTQ methods, calibration samples are drawn from the beginning of C4 training set (Raffel et al., 2020) in sequential order: 800 for Llama-3.2-3B-Instruct; 600 for Qwen2.5-7B-Instruct and L1-Max-Qwen-7B; 550 for Llama-3.1-8B-Instruct; 512 for DeepSeek-R1-Distill-Llama-8B and Qwen3-30B-A3B-Instruct-2507; and 400 for Qwen2.5-14B-Instruct. We also sample calibration sequences from the beginning of C4 training set in sequential order, using a sequence length of 4096 tokens for Qwen3-30B-A3B-Instruct-2507 and 2048 tokens for the other models. For the remaining hyperparameters, we follow the experimental settings recommended in prior work (Lee et al., 2023; Shao et al., 2024; Chen et al., 2025).

F. Learning Rate Sensitivity of LFQ

Table 11. Performance of Qwen2.5-7B-Instruct with LFQ under block-wise PTQ (FlexRound, OmniQuant, and Block-AP). Within each block-wise PTQ method, the best accuracy is shown in **bold** and the second-best is underlined. “W4” indicates 4-bit per-channel weight-only quantization. “lr” denotes the learning rate used in LFQ.

Method	# Bits	Language Modeling/Understanding		Text Generation	
		WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	MATH500 ($\uparrow$) (greedy)
Qwen2.5-7B-Instruct	BF16	6.85	73.49	70.79	74.2
FlexRound	W4	7.23	72.50	69.50	72.6
+ LFQ (lr = $5e-4$)	W4	7.20	72.41	<u>71.16</u>	<u>73.2</u>
+ LFQ (lr = $1e-3$)	W4	<u>7.21</u>	<u>72.48</u>	71.35	73.4
OmniQuant	W4	7.73	71.00	68.21	69.8
+ LFQ (lr = $1.5e-3$)	W4	<u>7.57</u>	70.93	69.87	<u>70.6</u>
+ LFQ (lr = $2e-3$)	W4	7.53	<u>70.99</u>	<u>69.50</u>	71.6
Block-AP	W4	7.87	69.60	66.73	68.0
+ LFQ (lr = $2e-5$)	W4	<u>7.77</u>	69.94	<u>68.02</u>	69.0
+ LFQ (lr = $3e-5$)	W4	7.75	<u>69.85</u>	68.76	<u>68.4</u>

As demonstrated in Table 11, LFQ consistently improves the generation quality of low-bit quantized LLMs regardless of the choice of learning rate.

G. Comparison of WikiText2 and C4 as calibration datasets for LFQ

Table 12. Performance of Qwen2.5-7B-Instruct with LFQ under block-wise PTQ (FlexRound, OmniQuant, and Block-AP). Within each block-wise PTQ method, the best accuracy is shown in **bold** and the second-best is underlined. “W4” indicates 4-bit per-channel weight-only quantization. The calibration dataset used in LFQ is either WikiText2 or C4.

Method	# Bits	Language Modeling/Understanding		Text Generation	
		WikiText2 ($\downarrow$)	MMLU ($\uparrow$)	IFEval ($\uparrow$) (greedy)	MATH500 ($\uparrow$) (greedy)
Qwen2.5-7B-Instruct	BF16	6.85	73.49	70.79	74.2
FlexRound	W4	7.23	72.50	69.50	72.6
+ LFQ (WikiText2)	W4	7.15	72.37	70.98	73.0
+ LFQ (C4)	W4	<u>7.21</u>	<u>72.48</u>	71.35	73.4
OmniQuant	W4	7.73	71.00	68.21	69.8
+ LFQ (WikiText2)	W4	7.47	70.92	69.87	<u>71.6</u>
+ LFQ (C4)	W4	<u>7.53</u>	<u>70.99</u>	69.50	71.6
Block-AP	W4	7.87	69.60	66.73	68.0
+ LFQ (WikiText2)	W4	7.73	<u>69.85</u>	68.39	<u>68.8</u>
+ LFQ (C4)	W4	<u>7.77</u>	69.94	<u>68.02</u>	69.0

Table 12 shows that LFQ enhances the generation quality of low-bit quantized LLMs irrespective of the choice of calibration dataset.